CSC_51053_EP, Exercise Session: Indexing, Query Evaluation

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

October 24, 2025

1 SQL queries: buying a gift together

We consider a database about users who pool money to buy someone a gift; the database also has information about possible gifts, and about what kind of gift each user thinks is best. *The database described below describes only one effort to buy such a gift.*

The tables are created through the following commands:

create table **guser**(uid int primary key, uname varchar);
– *people who pool money to buy a gift*
create table **category**(cid int primary key, cname varchar);
– *gift categories, e.g., 'sport', 'art', 'cooking' etc.*
create table **gift**(id int primary key, descr varchar, price int, cID int references category(cid));
– *possible gifts that could be bought*
create table **contribution**(uid int primary key, amount int not null);
– *each user may contribute (or not) an amount of money to the gift*
create table **preference**(uid int references guser(uid), cid int references category(cid), porder int not null check (1<=porder and porder<=3), primary key (uid, cid, porder));
– *each user may specify at most 3 preferences for the category of the gift to be bought. A user preference order (porder) of 1 is the option the user favors most; 2 is the "next" preference order, and 3 is the least favored preference order*

If a user u put a category c first in her preference order (porder=1), we view this as a *support score* of 3; similarly, porder=2 means a support score of 2, while porder=3 means a support of 1.

1. Write the SQL query Q1: for each category whose name starts with 'M', with non-zero total support across all users, find the category name and the total category support. Order the categories from the highest to the lowest total support.

   select c.cName, sum(4-p.porder) as totalSupport
   from preference p, category c
   where c.cname like 'M%' and p.cid=c.cid
   group by c.cid, c.cName
   order by totalSupport desc
   Physical plan (tree of physical operators): `https://explain.dalibo.com/plan/75ge614227e974g6`

   Where exactly in the query is the non-zero total support condition expressed?

   In the above query, it's in the join (a category without any preference has 0 total support). We could also have written "having totalSupport > 0" but the having clause would have been redundant.

2. Write the variant query Q2: for each category (including those with no support), find the category name and the total support. Provide one variant using an outerjoin, and another without.

   With outerjoin:
   select c.cName, sum(4- (case when (p.porder is null) then 4 else p.porder end)) as totalSupport
   from category c left outer join preference p on (c.cid=p.cid)
   group by c.cid, c.cName
   order by totalSupport desc
   Physical plan: `https://explain.dalibo.com/plan/fdf65888ee18c339`
   One can also use the function coalesce() as sum(4- (coalesce(p.porder,4))) to get 0 total support

```
                                QUERY PLAN
-------------------------------------------------------------------------------
 Seq Scan on gift g  (cost=70.88..95.00 rows=6 width=40)
   Filter: (price = $2)
   CTE affordable
     ->  Seq Scan on gift  (cost=38.26..62.38 rows=377 width=40)
           Filter: (price <= $0)
           InitPlan 1 (returns $0)
             ->  Aggregate  (cost=38.25..38.26 rows=1 width=8)
                   ->  Seq Scan on contribution  (cost=0.00..32.60 rows=2260 width=4)
   InitPlan 3 (returns $2)
     ->  Aggregate  (cost=8.48..8.49 rows=1 width=4)
           ->  CTE Scan on affordable  (cost=0.00..7.54 rows=377 width=4)
(11 rows)
```

Figure 1: A possible plan for Q4.

for null values.
Note that the form below is not correct:
select c.cName, sum(4-p.porder) as totalSupport
from Category c left outer join Preference p on (c.cid=p.cid)
group by c.cid, c.cName
order by totalSupport desc
For categories with no preference, the totalSupport computed here is null (arithmetic operations on null give null), not the desired 0.

Without outerjoin:
select c.cName, sum(4-p.porder) as totalSupport
from preference p, category c
where p.cid=c.cid
group by c.cid, c.cName
union
select c.cName, 0 as totalSupport
from Category c
where c.cid not in (select cid from preference)
order by totalSupport desc
Physical plan: `https://explain.dalibo.com/plan/g456386b8e75a534`

3. A gift is *affordable* if its price is less than the sum of all contributions. Write the query Q3: For each affordable gift, return the gift id, description and price.

   select id, descr, price
   from gift
   where price <= (select sum(amount) from contribution);

4. A gift makes best use of the money if the gift (i) is affordable, and (ii) has a price that is the closest to the sum of all contributions, among all the affordable gifts.

   (a) Write the query Q4: Find the id(s), description(s) and price of gift(s) making best use of the money.

   with
   affordable as (select id, descr, price from gift where price <= (select sum(amount) from contribution))
   select id, descr, price
   from gift g
   where g.price = (select max(price) from affordable);

   A possible physical plan for Q4, given by PostgreSQL, appears in Figure 1. Note the CTE

```
                                    QUERY PLAN
-------------------------------------------------------------------------------
 Seq Scan on gift g  (cost=63.34..87.46 rows=6 width=40)
   Filter: (price = $1)
   InitPlan 2 (returns $1)
     ->  Aggregate  (cost=63.33..63.34 rows=1 width=4)
           InitPlan 1 (returns $0)
             ->  Aggregate  (cost=38.25..38.26 rows=1 width=8)
                   ->  Seq Scan on contribution  (cost=0.00..32.60 rows=2260 width=4)
           ->  Seq Scan on gift  (cost=0.00..24.12 rows=377 width=4)
               Filter: (price < $0)
(9 rows)
```

Figure 2: A possible plan for the alternative formulation of Q4 without the WITH clause.

(common table expression) that builds the table affordable. A visualization appears at: `https://explain.dalibo.com/plan/2bdffc1c3bbac9h7`.

In general, a CTE is materialized temporarily, used in the query, and dropped after answering the query. The CTE is evaluated only once, even if the parent query refers several times to the CTE. This prevents expensive recomputations. However, CTEs also tend to limit the possibilities for the query optimizer to push down the predicates.

In our case, the plan is: scan gift, and filter the tuples by requiring that the price be equal to the variable (temporary result) $2; in turn, $2 is computed (bottom of the plan) by aggregating over affordable.

Depending on the statistics of the actual database, the plan may change, of course.

Can we write the same query without a WITH? What would be the impact on the physical plan?

Yes: just take the query defining affordable and nest it in the WHERE clause (or the FROM clause).

```
select id, descr, price
from gift g
where g.price = (select max(price)
                 from (select id, descr, price
                       from gift
                       where price < (select sum(amount) from contribution)) as affordable);
```

Figure 2 shows a corresponding alternative physical plan produced by PostgreSQL. There is no CTE any more. Now the sum of the contributions is in the computed field $0, while the highest affordable gift price is in $1.

A visualization (on a different database): `https://explain.dalibo.com/plan/4506328cgbad42e1`.

5. While searching for "the ideal gift to buy", we see there are two distinct ways to order them:

   - from the most expensive one (still within the budget), to the least expensive one; we prefer gifts that spend as much of the budget as possible;
   - according to their categories, from the category with most support to the category with least support.

   In general, there may be no single best gift, as one who is ranked at the top on one dimension may be ranked lower in the other dimension.

   We say a gift g_1 is *Pareto-optimal* if there exists no other gift g_2 that is better than g_1 on both dimensions.

   (a) Write the query Q5 that returns the IDs and descriptions of all the Pareto-optimal gifts. Propose a physical plan for it.

   Variant 1 We write the query as: find all the gifts that are (i) affordable, and (ii) not simultaneously price-dominated and category-dominated by any other:

```
– the gifts that are within budget:
with affordablegift as
(select g.id, g.cid, g.descr, g.price
from gift g
where g.price <= (select sum(amount) from contribution)),
– the support of each gift category
categorysupport as
(select c.cid, c.cName, sum(case when (p.porder is null) then 0 else 4-p.porder end) as totalSupport
from Category c left outer join Preference p on (c.cid=p.cid)
group by c.cid, c.cName)
– the main query block:
select a.id, a.descr
from affordablegift a, categorysupport csa
where a.cid=csa.cid
      and not exists(select * from affordablegift o, categorysupport cso
                     where o.cid=cso.cid and
                     ((cso.totalsupport>=csa.totalsupport and o.price> a.price)
                      or (cso.totalsupport>csa.totalsupport and o.price>=a.price));
```

Physical plan: `https://explain.dalibo.com/plan/2g1d935ec266f8d9`

Here, the Pareto-optimality is ensured by the "not exists" clause. The top-most operator Anti Join unnests the "where... not exists(...)" from the top-level query, just like an "where... exists(...)" is unnested into a join.

Propose one or two more variants of this query.

Variant 2 We could compute the ranking of each gift along each dimension (giftsInPriceOrder and giftsInSupportOrder), then join the result to associate the two ranks to each gift (giftsInBothOrders), finally decide based on these associated ranks if a gift is Pareto-optimal:

```
– the gifts that are within budget:
with affordablegift as
(select g.id, g.cid, g.descr, g.price
from gift g
where g.price <= (select sum(amount) from contribution)),
– the affordable gifts with the number of cheaper affordable gifts (higher is better):
giftsInPriceOrder as
(select g.id, count(*) as priceOrder
from affordablegift g, affordablegift o
where g.price >= o.price
group by g.id),
– the support of each gift category:
categorysupport as
(select c.cid, c.cName, sum(case when (p.porder is null) then 0 else p.porder end) as totalSupport
from Category c left outer join Preference p on (c.cid=p.cid)
group by c.cid, c.cName),
– the affordable gifts with the support of their category (higher is better):
giftsInSupportOrder as
(select g.id, count(*) as supportOrder
from affordablegift g, affordablegift o, categorysupport cs, categorysupport cso
where g.cid=cs.cid and o.cid=cso.cid and cs.totalSupport >= cso.totalSupport
group by g.id),
– gifts with both their dimensions: higher is better on each:
giftsInBothOrders as
(select gp.id, gp.priceOrder, gs.supportOrder
from giftsInPriceOrder gp, giftsInSupportOrder gs
where gp.id=gs.id)
```

```
gift=# explain select * from preference where exists (select * from category c where cid=c.cid);
                              QUERY PLAN
-----------------------------------------------------------------------
 Result  (cost=4.31..34.71 rows=2040 width=12)
   One-Time Filter: $0
   InitPlan 1 (returns $0)
     ->  Seq Scan on category c  (cost=0.00..25.88 rows=6 width=0)
           Filter: (cid = cid)
   ->  Seq Scan on preference  (cost=4.31..34.71 rows=2040 width=12)
(6 rows)

gift=# explain select * from preference;
                            QUERY PLAN
------------------------------------------------------------------
 Seq Scan on preference  (cost=0.00..30.40 rows=2040 width=12)
(1 row)
```

Figure 3: Plans for two equivalent queries.

```
select gbo.id, g.descr
from giftsInBothOrders gbo, gift g
where gbo.id=g.id
      and not exists(select * from giftsInBothOrders gbo2
                     where gbo2.id<>gbo.id
                           and ((gbo2.priceOrder>=gbo.priceOrder and
                                 gbo2.supportOrder>gbo.supportOrder)
                                or (gbo2.priceOrder>gbo.priceOrder and
                                    gbo2.supportOrder>=gbo.supportOrder));
```

Physical plan: `https://explain.dalibo.com/plan/a539b36g267fba03`.

Variant 3 One could also CREATE VIEWs instead of the WITH-introduced expressions in Variant 1.

- Recall that CREATE VIEW[1] just associates a view name to a query, so that the view can be used subsequently like a table.
- In contrast, if we create a materialized view[2], the view data is also computed and materialized. This is a good idea if the view is used frequently and it is costly to compute. However, a materialized view is also maintained by the system to keep it up to date when the data changes. Therefore, updates become slower in the presence of materialized views.

```
create view affordablegift as
(select g.id, g.cid, g.descr, g.price
from gift g
where g.price <= (select sum(amount) from contribution));

create view categorysupport as
(select c.cid, c.cName, sum(case when (p.porder is null) then 0 else p.porder end) as totalSupport
from Category c left outer join Preference p on (c.cid=p.cid)
group by c.cid, c.cName);
select a.id, a.descr
from affordablegift a, categorysupport csa
where a.cid=csa.cid and not exists(select * from affordablegift o, categorysupport cso where
o.cid=cso.cid and cso.totalsupport>csa.totalsupport and o.price>a.price);
```

6. *(Modeling)* Modify the schema described above so that it can capture *several* gift pools (e.g., in the style of the Leetchi platform). Describe briefly how the above queries Q1-Q5 could be re-stated in natural language (think of at least two variants).

   A solution is the following:

   - Add a table giftPool(gpid int primary key, description: varchar) such that each tuple in giftPool corresponds to one concrete gift to be bought. This can be refined, e.g., to state to whom the giftPool is for, when does it close etc.

[1] See `https://www.postgresql.org/docs/current/sql-createview.html`.
[2] See `https://www.postgresql.org/docs/current/sql-creatematerializedview.html`.

- To the table contribution, add an attribute gpid which is a foreign key into giftPool: it specifies to which gift pool each contribution is for.
- Similarly, to the table preference, add an attribute gpid which is a foreign key into giftPool: it specifies to which gift pool each user preference refers to.
- We may want to add a constraint stating that a user should not express preferences for a gift pool to which she does not contribute (how?); or, we may not add it, along the idea that the user may give an interesting information even if she cannot contribute financially.

The queries Q1-Q5 can be reformulated on this richer schema in two ways:

- For *a given gift pool*, e.g., gpid=5: in this case, instead of using contribution in the above tables, we need to use contribution joined with giftPool to make sure we only consider money for the given gift pool;
- For *each gift pool*: in this case, each query can be enhanced to add it an additional attribute gpid, to allow interpreting each results in the context of that pool.

2 B+ trees insertions and deletions

1. Consider a B+-tree whose maximum number of keys n in a node is 4. The tree is initially empty. Show the tree obtained after inserting *successively, one by one*, the keys $1^*, 2^*, \ldots, 13^*$.

   See Figure 4.

2. Now show the tree resulting from inserting these keys through bulk-loading.

   We sort all the entries, which we group immediately into three leaf nodes: $\boxed{1^*|2^*|3^*|4^*}$, $\boxed{5^*|6^*|7^*|8^*}$, $\boxed{9^*|10^*|11^*|12^*}$. The remaining value cannot be put into a leaf node $\boxed{13^*|\,|\,|}$ as this hypothetical node would have too few values. Thus, we build a root node $\boxed{5|9||}$ that has the first three children. Then we insert 13^* in r^*, the rightmost node above the leaf level – which in our case is the root. 13^* attempts to go in the 9 node which is too full; thus, this node splits, 11 goes into the parent which becomes $\boxed{5|9|11|}$, with the children: $\boxed{1^*|2^*|3^*|4^*}$, $\boxed{5^*|6^*|7^*|8^*}$, and also $\boxed{9^*|10^*||}$ on one hand and $\boxed{11^*|12^*|13^*|}$ on the other.

3. Delete the value 10^* in the B+ tree represented in Figure 5. Delete the value 3^* in the B+ tree represented in Figure 5. Delete 13^* from the B+ tree represented in Figure 5.

   Deleting 13^* does not modify the tree structure. See Figure 6 for the other operations.

4. Delete the values 13^*, 10^* and 3^* *one after another* from the tree in Figure 5 (thus each deletion applies on the result of the previous one).

   Deleting 13^* does not change the tree structure. The impact of deleting 10^* and 3^* after 13^* is shown in Figure 7.

References

[1] Raghu Ramakrishnan and Johannes Gehrke. *Database management systems (3rd ed.)*. McGraw-Hill, 2003.

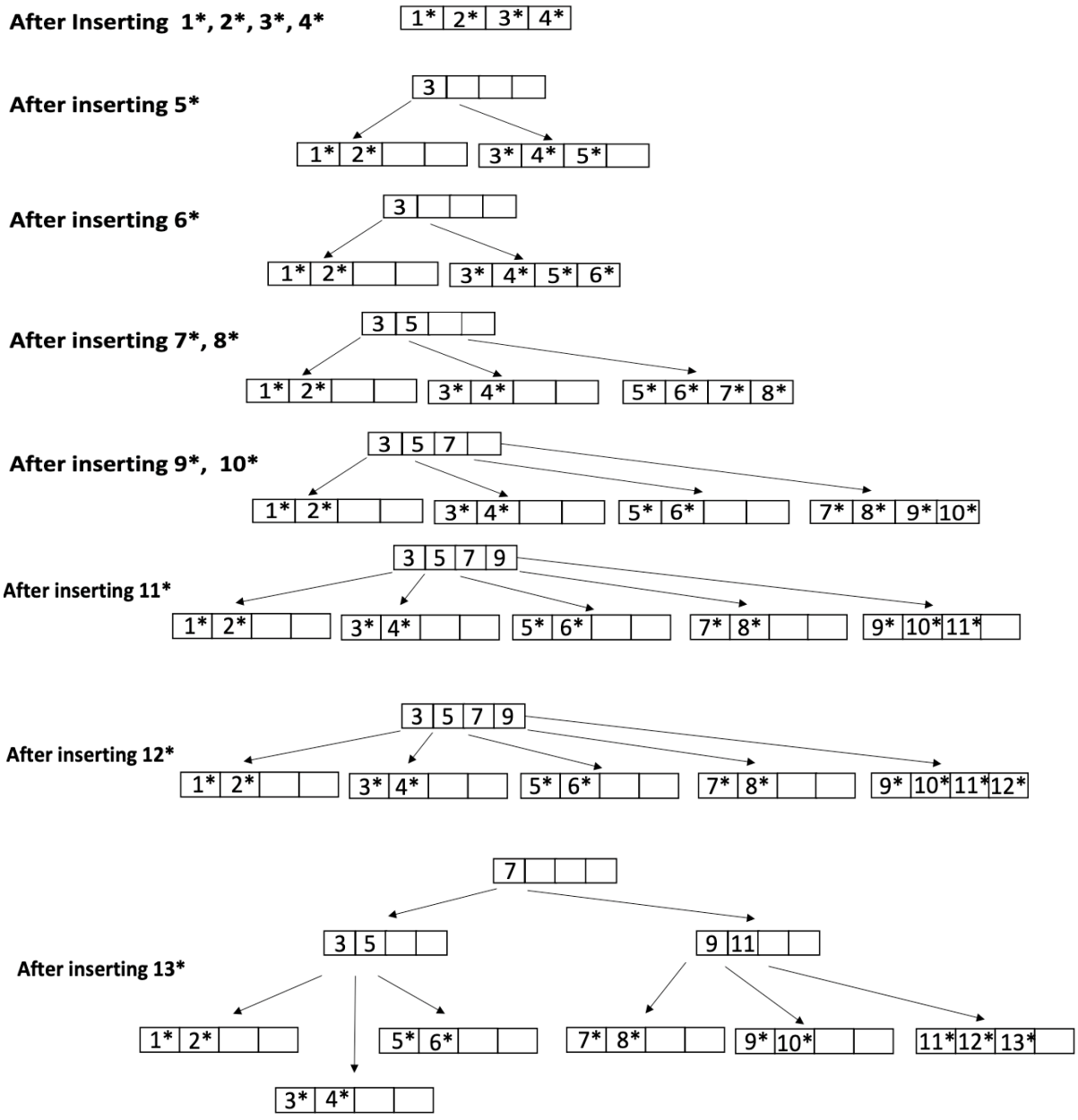


Figure 4: Inserting 1^* up to 13^* in a B+ tree.

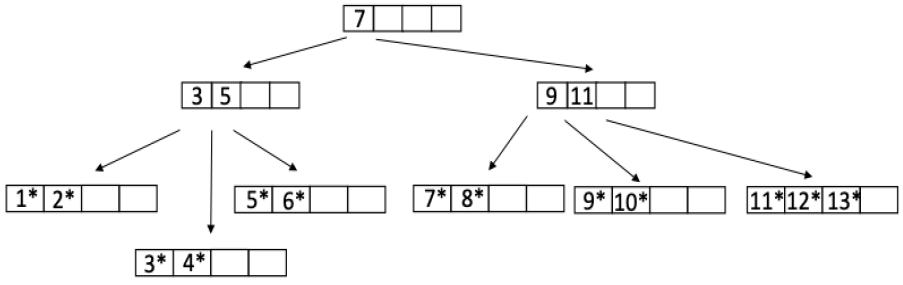


Figure 5: Tree for B+ tree deletions.

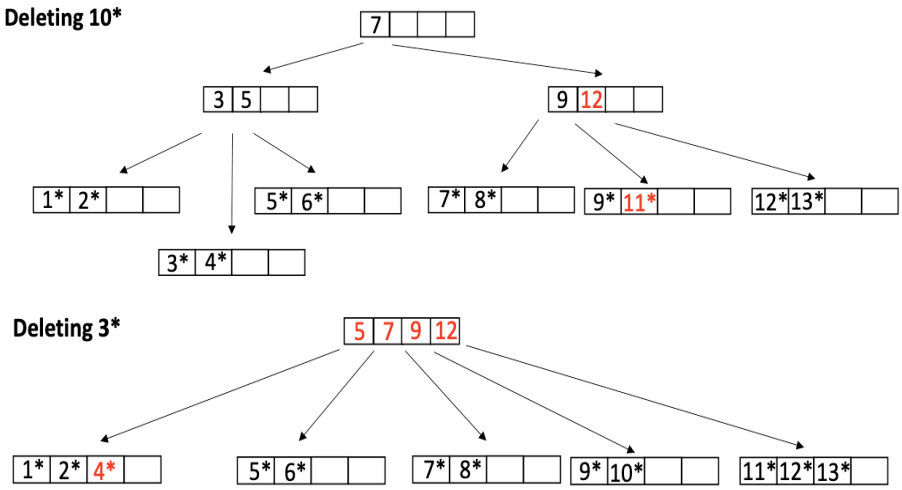


Figure 6: Impact of *alternative* deletions from B+ tree in Figure 5.

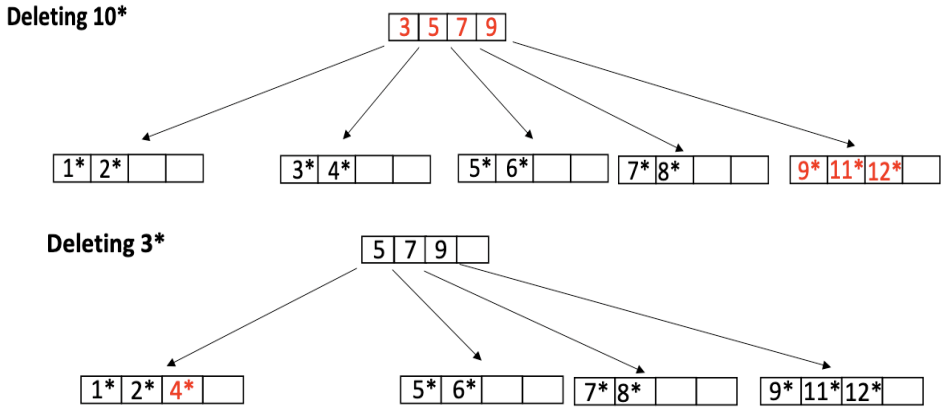


Figure 7: Impact of *successive* deletions from the B+ tree in Figure 5 (after deleting 13^*).